How to upgrade a Kogger device — the wire procedure

A step-by-step guide for anyone writing or driving an upgrader: what to send, what to wait for, what to do when it does not arrive.

Derived from host code (`src/device/dev_driver.cpp`, `src/id_binnary.{h,cpp}`, `src/proto_binnary.h`). Firmware sources were not read; device behaviour below is what the host depends on, plus what the firmware side has stated. Section 8 adds figures measured on a real upgrade.

> **Scope — and one hard prohibition**
>
> This document describes **only what the host does**.
>
> **The `.ufw` encryption method and the internal design of the firmware are not disclosed here and must not be disclosed under any circumstances** — not in this document, not in correspondence, not in answers to follow-up questions.
>
> Everything below is open: the exchange protocol lives in the KoggerApp sources, which are public.

Before you start

1. **A full-duplex link.** The procedure is request/answer throughout; a listen-only or replayed stream cannot flash anything.
2. **The device identified and answering**, so you know its `route` (bus address — a Recorder answers on its own, a bare sonar on 0). Every frame you send goes to that route.
3. **The image in memory.** Nothing in the protocol describes or checks the file: no header, no board id, no length, no CRC. **You cannot rely on the bootloader to reject a wrong image** — whether it validates before jumping depends on the bootloader. If you have any way to check the file against the target board, do it before step 1, not after.
4. **Know your window.** From the moment the bootloader starts you have **~3 s** (**5 s** on the Recorder, an exception granted for USB compatibility) to get the *first* chunk in. Miss it and the device runs the old image — no error, nothing flashed.
5. **Expect the transport to disappear.** On a USB-VCP device the reset detaches the USB device and its port vanishes for a re-enumeration, typically 1–2 s of your window. Whatever holds the image and the retry has to survive that; do not tie it to a per-connection object.

Step 1 — Reset into the bootloader

Send `ID_BOOT v0` `SETTING` with the key.

The ack may arrive after the device is already gone, and answers to requests you sent *before* the reset can arrive *after* it. **Treat everything received in that gap as stale** — in particular an `ID_VERSION` answer that says "firmware, healthy", which is a pre-reset leftover, not news.

Step 2 — Get the link back

Reopen the transport as soon as it exists again, and keep retrying: the port may be absent for a second or two. Do not spend a single reconnect attempt during the gap and give up — that is the classic way to burn the whole window.

KoggerApp also keeps its baudrate search active for the duration of an upgrade, so a bootloader that comes up at a different rate is still found. If you know your bootloader's rate, use it; if not, search.

Step 3 — Prove you are talking to a bootloader

On a repeating tick (KoggerApp uses 200 ms):

1. Send `ID_MARK v0` `SETTING` with the key. The device answers with **bit 6 of** `mode` **set** in its frames; that bit is your "this device is alive in this session" signal.
2. Request `ID_VERSION` (`GETTING` v0, v1, v2).

Proceed only when **both** hold:

- frames are coming back with the mark bit set, and
- `ID_VERSION v2` reports a known board with `bootMode != 0` (`1` = bootloader, `0` = firmware).

`bootMode` is the only thing on the wire that distinguishes a bootloader from a running firmware. While you are being told `0`, keep re-requesting the version — do not start sending chunks. A bootloader that answers no v2 at all leaves `bootMode` unknown; then the best you have is a short time guard against the stale answer from step 1.

If the mark bit is missing from the device's frames, restart this step: something reset the session.

Step 4 — Send the image

Chunks are `ID_UPDATE v0` `SETTING`: `U2 packetNum` followed by up to **96 image bytes**. Numbering starts at **1** and increases by exactly 1. 96 B is the recommended size for cross-device compatibility, not a protocol limit — the frame would carry up to 255, but a larger chunk risks a device that will not take it.

Send one chunk, wait for the answer, send the next. Optional pacing: KoggerApp keeps one extra chunk in flight from chunk 3 onward.

The device answers in one of two dialects, and **you must handle both**:

Dialect	Answer	What you learn
regular	`ID_UPDATE v0` `CONTENT`: `U2 lastNumMsg, U4 lastOffset, U1 type, U1 rcvNumMsg`	its cursor and a condition code
legacy	a bare ack: `CONTENT resp=1`, code `respOk`	only that the chunk arrived

Assume legacy until a progress frame proves otherwise — that is what KoggerApp does, and it never goes back once it has seen one. In legacy mode you get no cursor feedback at all, so a lost chunk is invisible and unrecoverable; the ack and the frame checksum are the whole safety net.

Condition codes in the regular dialect:

`type`	Means	Do
0	chunk accepted	send the next chunk
1	the device is **behind** your cursor	log it and continue (KoggerApp does not reposition)
2	the device is **ahead** / expects another point	set `next = lastNumMsg + 1`, `offset = lastOffset`, continue from there
> 2	fatal	stop, report failure, and release the transport

`rcvNumMsg` can be ignored.

When an answer does not come (KoggerApp waits 2 s), resend **the same number at the same offset** and do not advance. The device stores a chunk only when its number is `lastNumMsg + 1`, so a cursor that advanced over a lost chunk produces a corrupt image *while the transfer still reports success* — the one failure that no later step catches. **Bound the retries** and abort when the bound is hit; 5 is a reasonable limit.

Nothing here races a device clock: the window from "Before you start" is switched off as soon as the first chunk lands. A stall mid-transfer is bounded only by your own timeout.

Step 5 — Run the new image, then check it

Send `ID_BOOT v1` `SETTING` with the key when the image is exhausted.

There is no completion handshake and no image checksum: the transfer is over because you ran out of bytes. Verify out of band — re-request `ID_VERSION v2` after the device comes back and compare `fwVer` / `fwVerMinor` against what you flashed.

If it goes wrong

Symptom	Meaning	Action
no frames after the reset	port still gone, or bootloader not up	keep reopening; give up on a deadline (8 s is reasonable), never silently
frames arrive, `bootMode` stays `0`	the window lapsed and the old firmware is running	abort; the device is healthy, try again
frames arrive with no mark bit	session not established	restart step 3
answers stop mid-transfer	link glitch or device fault	resend the same number/offset, bounded, then abort
`type = 1` repeatedly	your cursor is ahead of the device's	a device that keeps waiting for a missing number is a hang the host cannot see; abort on a bound rather than looping
`type > 2`	device-side fatal	abort
flash "succeeded" but the version did not change	chunks were lost, or the bootloader rejected the image	re-flash; step 5's version check is what catches this

Always leave the flow explicitly, on failure as well as success, releasing whatever transport state the upgrade reserved. An upgrader that can hang keeps the port reserved for the rest of the session.

Reference

Frame (KP1) — `0xBB 0x55 | route | mode | id | len | payload | check1 check2`

- `mode` = `type & 0x3` | `ver << 3` | `mark << 6` | `resp << 7`; `type`: `CONTENT = 1`, `SETTING = 2`, `GETTING = 3`
- `check1/check2` = Fletcher16 over bytes `[2 .. 6+len)`; `len` is one byte, so payload ≤ 255 B
- command = `SETTING`; ack = `CONTENT resp=1` with `U1` code + `U2` echo of the commanded frame's checksum (use it to match acks to requests); data = `CONTENT resp=0`
- codes: `0 respNone`, `1 respOk`, then `respErrorCheck/Payload/ID/Version/Type/Key/Runtime`

IDs used

ID	ver / type	Dir	Payload
`ID_BOOT 0x24`	v0 `SETTING`	H→D	`U4 key` — reset into the bootloader
`ID_BOOT 0x24`	v1 `SETTING`	H→D	`U4 key` — run the flashed image
`ID_UPDATE 0x25`	v0 `SETTING`	H→D	`U2 packetNum` + ≤96 image bytes
`ID_UPDATE 0x25`	v0 `CONTENT`	D→H	`U2 lastNumMsg, U4 lastOffset, U1 type, U1 rcvNumMsg`
`ID_VERSION 0x20`	v2	both	`U1 bootMode, U1 boardMinor, U1 board, U1 bootVerMinor, U1 bootVer, U2 -, U1 fwVerMinor, U1 fwVer`
`ID_MARK 0x21`	v0 `SETTING`	H→D	`U4 key` — ask for the mark bit

`key` is one fixed `U4` — `0xC96B5D4A`, the constant `IDBin::m_key` — carried by `ID_BOOT`, `ID_MARK`, `ID_FLASH` and the setup IDs; `ID_UPDATE` carries none. If this document and the sources ever disagree, believe the sources: that is where the value is maintained. `ID_FLASH 0x23` is the **settings** store (flash/restore/erase), not the image — it plays no part here.

Numbers

Quantity	Value	Whose
window, bootloader start → first chunk (then off)	~3 s; Recorder 5 s	device
USB-VCP re-enumeration after the reset	~1–2 s	device / OS
handshake deadline, reset → first bootloader answer	8 s	host choice
per-chunk answer timeout / resends before abort	2 s / 5	host choice
tick driving the state machine / delay before the first chunk	200 ms / 100 ms	host choice
chunk payload	96 B recommended	compatibility
max image	65535 chunks ≈ 6 MiB	protocol (`U2` numbering)

Host deadlines are all longer than the device window by design: they release the flow, they do not save the window. What keeps chunks off a device that has already returned to firmware is the `bootMode` check in step 3, not any clock.

The whole procedure in one loop

```
send(BOOT, v0, SETTING, key)                       # step 1
t0 = now
while bootMode != BOOTLOADER:                      # steps 2–3
    if now − t0 > HANDSHAKE_DEADLINE: fail("bootloader did not answer")
    reconnect_if_needed()
    send(MARK, v0, SETTING, key)
    request(VERSION, v0..v2)
    pump(200ms)                                    # a frame without the mark bit restarts this

num, off = 1, 0                                    # step 4
send(UPDATE, v0, SETTING, num, image[off:off+96]); num, off = num+1, off+96
while off < len(image):
    answer = wait(UPDATE, PACKET_TIMEOUT)
    if answer is None:
        if resends++ >= RESEND_LIMIT: fail("no answer")
        resend(num−1, off−96); continue            # same number, same offset
    resends = 0
    if answer.type > 2: fail("device reported a fatal condition")
    if answer.type == 2: num, off = answer.lastNumMsg + 1, answer.lastOffset
    send(UPDATE, v0, SETTING, num, image[off:off+96]); num, off = num+1, off+96

send(BOOT, v1, SETTING, key)                       # step 5
request(VERSION, v2); check(fwVer)
```

8. Measured on a real upgrade

Trace of an actual firmware update, 18 August 2026, over USB-UART (CP2105), captured from KoggerApp with per-packet instrumentation. Figures for calibration, not limits.

Quantity	Value
Image size	43,900 bytes
Body packets	458 × 96 bytes
Reset → first device answer	**1,732 ms**
Body transfer	4,370 ms
Total, reset → run command	6,102 ms
Throughput	≈ 9.8 kB/s
Per-chunk interval	9.6 ms mean (3…79)
Retransmits	0
`type` 1 or 2 events	0

Three observations worth carrying into an implementation.

The device answered the legacy dialect throughout. All 457 replies were a bare `respOk`; not one progress frame arrived, so the whole `type` / reposition branch never executed. This confirms the advice in step 4 from the other direction: **legacy is not a rare fallback, it is what you should expect to meet first.**

1,732 ms covers reset *and* USB re-enumeration together, not the reboot alone. Treat it as a sample, not a specification — it is well inside the ~3 s window, but nothing guarantees it. This is why step 2 says reconnect rather than sleep.

The double send at chunk 3 is visible in traces. Packet numbers run `2, 4, 5, 6…` because chunk 3 goes out inside the same operation as chunk 2. It is the optional pacing from step 4, not a numbering fault — worth knowing when diffing your exchange against KoggerApp's.

A note on versions

The retry bound and handshake deadline in the tables above are **host choices, and good ones**, but they are not present in every KoggerApp build: in the upstream `develop` checkout used for the measurements (`455b049e`), the timeout retransmit is **unbounded** and there is no handshake deadline.

Do not copy that. An unbounded retry is a hang that holds the port for the rest of the session, which is exactly what the "If it goes wrong" table warns against.